

ROUTEPILOT — DAILY FIELD VISIT PLANNER AND ROUTE OPTIMIZATION SYSTEM

A Dataset-Driven Decision-Support Platform for Constrained Multi-Vehicle Field Operations

ABSTRACT

In banking recovery, microfinance credit operations, utility field servicing, and supply chain logistics, field collection teams are charged with visiting delinquent customer accounts across large metropolitan areas. Daily scheduling has historically relied on manual heuristics, spreadsheet guesswork, or static territorial slicing. Consequently, enterprises face severe operational inefficiencies: uncoordinated crisscrossing journeys driving up fuel expenditures, missed borrower availability windows leading to wasted visits, non-adherence to mandatory Promise-to-Pay (PTP) commitments, and frequent violations of executive labor shifts and vehicular range capacities.

This project presents **RoutePilot**, a dataset-driven decision-support platform that deterministically formulates and solves the **Multi-Vehicle Routing Problem with Time Windows and Priority Commitments (m-VRPTW-P)**. RoutePilot ingests raw operational CSV datasets (`customers.csv` and `executives.csv`), conducts pre-flight schema audits, models urban road winding behavior through geodesic Haversine approximations, and executes a multi-stage optimization pipeline combining **Mandatory PTP Precedence**, **Marginal Priority Greedy Insertion**, and **2-Opt Edge-Exchange Local Search**.

The system is decoupled into two role-isolated operational workspaces: an **Operations Manager / Administrator Workspace** providing diagnostic visual analytics, interactive Leaflet geospatial route rendering, and audit logging; and a **Field Executive Cockpit** providing state-driven visit lifecycle management, turn-by-turn navigation dispatch, and real-time payment collection capture. When evaluated against an operational benchmark dataset of 200 customers and 6 field executives localized to the Hyderabad urban area, RoutePilot achieved **100.0% scheduling of all mandatory Promise-to-Pay commitments (49/49)**, scheduled 74 visits across 447.4 km of total fleet transit, generated explainable diagnostic reasons for all unscheduled accounts, and registered zero constraint breaches across 13 hard physical rules within a solver runtime of 5.44 seconds.

Keywords: Vehicle Routing Problem with Time Windows (VRPTW), Combinatorial Optimization, Heuristic Algorithms, Field Debt Collections, 2-Opt Local Search, Spatial Analytics, Decision Support Systems.

TABLE OF CONTENTS

- **Abstract**
- **List of Figures**
- **List of Tables**
- 1. Introduction**
 - 1.1 Background
 - 1.2 Problem Statement
 - 1.3 Motivation
 - 1.4 Objectives
 - 1.5 Scope and Deliverables
- 2. Existing System Analysis**
 - 2.1 Existing Operational Processes
 - 2.2 Operational Deficiencies
 - 2.3 Algorithmic and Structural Limitations
- 3. Proposed System: RoutePilot**
 - 3.1 Overview of RoutePilot
 - 3.2 Core Architectural Features
 - 3.3 Key Operational Advantages
- 4. Requirement Analysis**
 - 4.1 Functional Requirements
 - 4.2 Non-Functional Requirements
 - 4.3 Hardware Requirements
 - 4.4 Software Requirements
- 5. System Design & Architecture**
 - 5.1 Layered System Architecture
 - 5.2 End-to-End Data Flow
 - 5.3 Modular Subsystem Decomposition
 - 5.4 Relational Data Model & Schema Design
 - 5.5 User Roles and Access Control Matrix
- 6. Methodology & Mathematical Formulation**
 - 6.1 Data Ingestion & Single Source of Truth Pipeline
 - 6.2 Pre-Flight Validation Engine

- 6.3 Mathematical Formulation of m-VRPTW-P
- 6.4 Geodesic Distance and Kinematic Travel Model
- 6.5 The 13 Hard Physical Constraints Framework
- 6.6 Multi-Stage Heuristic Optimization Engine
- 6.7 2-Opt Sequence Refinement
- 6.8 Explainable Exception Handling

7. Implementation Details

- 7.1 Backend API Engine (FastAPI & Python)
- 7.2 Database & Persistence Layer (SQLAlchemy & SQLite)
- 7.3 Frontend Client Architecture (React 18 & TypeScript)
- 7.4 Interactive Map & Spatial Rendering (Leaflet)
- 7.5 Real-Time Field-to-Manager Synchronization Engine

8. User Interface & Interaction Design

- 8.1 Operations Manager Cockpit
- 8.2 Data Center & Ingestion Pipeline View
- 8.3 Daily Planning Setup & Algorithm Comparator
- 8.4 Route Management & Route Detail Itinerary
- 8.5 Interactive Spatial Map View
- 8.6 Exceptions & Diagnostic Queue
- 8.7 Historical Plan & Compliance Exports
- 8.8 Field Executive Mobile Cockpit

9. Experimental Testing & Results

- 9.1 Test Strategy & Verification Framework
- 9.2 Comprehensive CSV Ingestion Matrix Tests
- 9.3 Physical Constraint Enforcement Test Cases
- 9.4 Regression Verification of Standard Scenarios

10. Performance Evaluation & Comparative Analysis

- 10.1 Benchmark Test Configuration (200 Customers, 6 Executives)
- 10.2 Metric Comparison: Baseline Sequential vs. Smart Greedy + 2-Opt
- 10.3 Route Distribution & Fleet Balance Analysis
- 10.4 Computational Complexity & Runtime Scalability

11. System Limitations

- 11.1 Methodological Limitations

- 11.2 Infrastructure & Environmental Assumptions

12. Future Enhancements & Engineering Roadmap

- 12.1 Real-Time Road Topology & Dynamic Traffic Matrix
- 12.2 Machine Learning for Predictive Repayment Likelihood
- 12.3 Offline Progressive Web Application (PWA) Architecture
- 12.4 Automated Fleet Failover & Dynamic Mid-Day Re-routing

13. Conclusion

- **References**
- **Appendix: Data Schemas & API Endpoints**

LIST OF FIGURES

- **Figure 1.1:** Conceptual Vehicle Routing Problem with Time Windows (VRPTW).
- **Figure 5.1:** Layered High-Level System Architecture of RoutePilot.
- **Figure 5.2:** End-to-End Operational Lifecycle Data Flow Diagram.
- **Figure 5.3:** Entity-Relationship Diagram (ERD) of Relational Persistence Schema.
- **Figure 6.1:** Multi-Stage Solver Architecture (PTP Placement $\rightarrow$ Greedy Insertion $\rightarrow$ 2-Opt Refinement).
- **Figure 6.2:** 2-Opt Edge-Exchange Sequence Inversion Mechanism.
- **Figure 7.1:** Real-Time Bidirectional Event Synchronization Flow between Field Executive and Operations Manager.
- **Figure 8.1:** Operations Manager Daily Overview Dashboard.
- **Figure 8.2:** Interactive Leaflet Route Map Visualizing Executive Itineraries in Hyderabad Urban Zone.
- **Figure 8.3:** Field Executive Mobile Cockpit and Visit State Machine.
- **Figure 10.1:** Metric Comparison Chart: Baseline Sequential vs. Smart Greedy + 2-Opt.

LIST OF TABLES

- **Table 4.1:** Hardware Specifications.
- **Table 4.2:** Software Stack Specifications.
- **Table 5.1:** User Roles and Operational Permissions Matrix.
- **Table 6.1:** Schema Specification for `executives.csv`.
- **Table 6.2:** Schema Specification for `customers.csv`.

- **Table 6.3:** Summary of the 13 Physical Constraints Enforced by RoutePilot.
 - **Table 9.1:** Summary of Pytest Verification Suite Results (22 Passed Tests).
 - **Table 9.2:** Detailed Test Case Matrix: Inputs, Expected Results, and Observed Results.
 - **Table 10.1:** Performance Benchmark Comparison: Baseline Sequential vs. Smart Greedy 2-Opt (200-Customer Roster).
 - **Table 10.2:** Detailed Route Breakdown for All 6 Field Executives on Active Operational Plan.
-

CHAPTER 1: INTRODUCTION

1.1 Background

In modern banking, Non-Banking Financial Companies (NBFCs), microfinance institutions, telecommunications, and field utility enterprises, in-person customer visits are critical for loan servicing, debt recovery, and physical verification. When borrowers default or enter delinquency, financial institutions dispatch field debt collection executives to negotiate repayment schedules, collect funds, or establish physical contact.

These operations are inherently complex. A financial institution typically manages thousands of accounts spread across broad urban and suburban territories. Each customer account exhibits distinct delinquency levels, balances, and operational urgency. Crucially, debt collection is governed by legal, regulatory, and practical operational constraints: borrowers are only available at specific hours (e.g., before leaving for work or during evening hours), field representatives must travel on two-wheelers with limited fuel capacity, and labor regulations mandate strict daily shift limits.

1.2 Problem Statement

Given an operational date (D) , a set of available field collection executives (E) , and a portfolio of overdue delinquent accounts (C) , the objective is to determine: 1. **Assignment**: Which field executive $(e \in E)$ should visit which customer $(c \in C)$? 2. **Sequencing**: In what exact chronological order should each executive execute their assigned visits? 3. **Feasibility**: How can the schedule satisfy customer-specific time windows, expected service durations, executive shift start/end times, maximum visit counts, and total daily distance limits while ensuring that every executive starts and finishes their shift at their designated home depot? 4. **Value Optimization**: How can the system maximize debt recovery prioritization and guarantee 100% protection for accounts with active payment promises (Promise-to-Pay), while minimizing unproductive transit mileage?

Historically, dispatch leads solve this manually using wall maps, intuition, and spreadsheets. Manual routing leads to substantial inefficiencies: multiple executives crisscross each other in the same neighborhoods, executives arrive after customers have left for work, high-value payment commitments are overlooked, and drivers exceed daily fatigue thresholds.

1.3 Motivation

The motivation behind **RoutePilot** is to bridge the divide between theoretical operations research and practical field operations. While traditional academic Vehicle Routing Problem (VRP) literature often relies on synthetic graphs and uniform depots, real-world field operations require a **dataset-driven decision-support tool**. The motivation is to construct an end-to-end platform that: * Ingests dynamic, unpredictable daily CSV files without code alterations; * Models the real geography of a dense Indian metropolitan region (Hyderabad Urban Zone); * Enforces strict business invariants (especially Promise-to-Pay protection); * Provides an interface that bridges executive management oversight with mobile field execution.

1.4 Objectives

The primary engineering and algorithmic objectives of RoutePilot are: 1. **Dynamic Dataset Ingestion**: Enable automated parsing, normalization, and persistence of daily customer lists and executive rosters without static assumptions about customer counts or fleet sizes. 2. **Pre-Flight Data Auditing**: Validate geographic coordinates, time formats, chronological window validity, and numeric ranges, issuing structured diagnostics instead of failing silently. 3. **PTP Protection**: Ensure that all feasible accounts with explicit Promise-to-Pay commitments (`ptp_today = 1`) are scheduled with strict priority. 4. **Heuristic Route Optimization**: Implement an objective-driven solver that balances priority value maximization against transit distance minimization using a balanced parameter λ . 5. **Physical Constraint Guarantee**: Strictly enforce 13 hard operational constraints (capacities, time windows, shift bounds, early-arrival waiting, range caps, and depot return). 6. **Geospatial Visualization**: Render computed routes, sequential stop markers, executive paths, and customer distribution on an interactive Leaflet map. 7. **Operational Decision Support**: Provide dispatch managers with transparent diagnostic visibility into unscheduled accounts (explainable rejection reasons). 8. **Field Execution Synchronization**: Provide field representatives with a mobile-responsive interface to execute stops, record payments, and reflect real-time progress back to the manager's console.

1.5 Scope and Deliverables

The scope of this project encompasses: * A mathematical solver written in Python, featuring Haversine road circuitry approximations and a 2-Opt local search engine; * A backend REST API developed with FastAPI and SQLite/SQLAlchemy; * A frontend web application constructed using React 18, TypeScript, and Tailwind CSS; * A dual-workspace architecture serving Operations Managers and Field Executives; * An automated test suite validating algorithmic correctness across 22 verification test scenarios.

The operational geographic scope is configured to the **Hyderabad Urban Region, Telangana, India**, running on **Asia/Kolkata (IST)** time using standard **12-hour AM/PM** temporal conventions.

CHAPTER 2: EXISTING SYSTEM ANALYSIS

2.1 Existing Operational Processes

In conventional field collection setups, operations are conducted through a decentralized, spreadsheet-driven workflow: 1. At the beginning of each business day, core banking servers generate raw dumps of delinquent accounts. 2. Operations leads partition accounts across executives based on broad postal pin codes or informal territorial rules (e.g., “Executive 1 covers Secunderabad, Executive 2 covers Kukatpally”). 3. Each executive receives a printed visit sheet or an unsequenced spreadsheet list. 4. The field executive decides their own travel sequence based on personal familiarity with roads.

2.2 Operational Deficiencies

This manual paradigm suffers from four severe operational flaws: * **Territory Overlap and High Fuel Burn:** Pin-code partitioning fails along neighborhood borders. Multiple executives frequently travel along the same arterial roads, duplicating travel distance. * **High Rate of Missed Time Windows:** Borrowers in urban settings frequently stipulate narrow availability windows (e.g., 09:00 AM – 11:30 AM). Executives operating without chronological routing arrive after the customer has departed, resulting in a wasted visit (“Door Locked / Customer Unavailable”). * **SLA Breaches on Promised Payments (PTP):** Delinquent borrowers who have promised to make payment on a specific date are the highest-probability recovery opportunities. Under manual sorting, executives prioritize geographically adjacent non-PTP accounts over urgent PTP accounts, resulting in broken promises and capital loss. * **Driver Fatigue and Overtime Breaches:** Without algorithmic travel-time calculation, executives are assigned unrealistic workloads, forcing them to work past legal shift hours or abandon stops midway.

2.3 Algorithmic and Structural Limitations

From a computer science perspective, the existing process fails because the underlying combinatorial search space is intractable for human intuition. A scenario with 6 executives and 200 customers has an astronomical configuration space of potential assignments and permutations ($\mathcal{O}(|E|^{|C|} \times |C|!)$). Manual dispatchers are incapable of evaluating trade-offs between early-arrival waiting times, service durations, and cumulative travel distances.

CHAPTER 3: PROPOSED SYSTEM: ROUTEPILOT

3.1 Overview of RoutePilot

RoutePilot is an algorithmic, dataset-driven daily visit planning platform designed to replace manual scheduling with an end-to-end mathematical optimization engine and dual-workspace operational system.

RoutePilot treats daily CSV datasets as the **Single Source of Truth**. Whether a manager uploads 5 customers for 1 executive or 200 customers for 6 executives, the system dynamically parses the files, validates operational feasibility, constructs optimized routes using a **Smart Greedy + 2-Opt Local Search algorithm**, and distributes the resulting plan to the field.

ROUTEPILOT PLATFORM	
Operations Manager Workspace	Field Executive Workspace
- Ingest & Validate Daily CSVs	- Locked Daily Shift Cockpit
- Compare Baseline vs 2-Opt	- Guided Visit State Machine
- Inspect Interactive Map	- Direct Google Maps Navigation
- Exceptions & Diagnostic Queue	- Capture UPI/Cash Collections
- Export Compliance Reports	- Real-Time Audit Replication

3.2 Core Architectural Features

- 1. Dynamic Dataset-Driven Engine:** Zero reliance on hardcoded mock arrays. Roster sizes, coordinates, shift times, and customer distributions are derived dynamically from uploaded files.
- 2. 100% PTP Protection Architecture:** Dedicated two-pass solver pipeline guaranteeing that high-value promise-to-pay accounts receive primary placement before standard accounts are evaluated.
- 3. 13 Hard Physical Constraints Framework:** Comprehensive validator rejecting any sequence violating vehicle capacity, shift return deadlines, customer time windows, or depot origins.
- 4. Explainable Rejection Engine:** Translates algorithmic exclusions into human-readable business diagnostics (e.g., *“Shift capacity reached across fleet”*, *“Time window incompatible with transit time”*).
- 5. Bidirectional Real-Time Synchronization:** When an executive marks a visit started, completed, or records collected funds on mobile, the Operations Manager workspace reflects the updated status instantly.

3.3 Key Operational Advantages

- Fuel and Mileage Reduction:** 2-Opt edge swaps eliminate route self-intersections and backtracking.
- Higher Recovery Yield:** Priority-aware objective formulation ensures maximum overdue balances are contacted within operating constraints.

- **Regulatory and Shift Compliance:** Shift-end deadlines are strictly guaranteed; no executive is scheduled to return after their contracted shift end.
 - **Operational Agility:** Allows managers to upload new daily datasets each morning and produce field-ready schedules in seconds.
-

CHAPTER 4: REQUIREMENT ANALYSIS

4.1 Functional Requirements

1. **FR-1: CSV Data Ingestion:** The system shall accept `executives.csv` and `customers.csv` files via HTTP multipart upload, supporting variable row counts and delimiter structures.
2. **FR-2: Pre-Flight Data Validation:** The system shall validate column schemas, verify latitude `\([-90, 90]\)` and longitude `\([-180, 180]\)` bounds, ensure chronological time window order (`window_start` < `window_end`), and verify positive numeric values.
3. **FR-3: Mandatory PTP Scheduling:** The optimization engine shall prioritize customers with `ptp_today = 1` over all non-PTP accounts.
4. **FR-4: Feasible Route Generation:** Generated itineraries shall adhere to all 13 physical constraints including shift limits, vehicle range caps, and customer time windows.
5. **FR-5: Multi-Algorithm Execution:** The system shall allow managers to run and compare Baseline Sequential, Smart Greedy, and Smart Greedy with 2-Opt local search.
6. **FR-6: Interactive Spatial Visualization:** The system shall plot executive routes, depot pins, customer stops, and sequence numbers on a geographic street map.
7. **FR-7: Explainable Exception Reporting:** All unscheduled customers must be categorized with diagnostic reasons explaining why scheduling was infeasible.
8. **FR-8: Real-Time Field Execution:** Field executives shall be able to progress stops through a state machine (`En Route` `\(to\)` `Arrived` `\(to\)` `In Visit` `\(to\)` `Completed`), recording payment outcomes and notes.
9. **FR-9: Role-Isolated Authentication:** The system shall enforce JWT-based authorization restricting field executives to their own assigned route while granting dispatch leads full operational control.
10. **FR-10: Compliance and MIS Export:** The platform shall export generated plans in standard CSV formats (`plan.csv` , `skipped_customers.csv` , `metrics.csv`).

4.2 Non-Functional Requirements

1. **NFR-1: Performance & Latency:** The optimization solver must complete route generation for a 200-customer / 6-executive dataset within 10 seconds.
2. **NFR-2: Data Integrity & Versioning:** Re-optimizing a plan must create an incremented plan version without overwriting prior execution history.
3. **NFR-3: Reliability & Zero Data Loss:** Invalid CSV rows must be flagged with diagnostic warnings; clean records must be stored and preserved without field loss.
4. **NFR-4: Usability & Human Factors:** The interface must adhere to professional UX standards: 12-hour AM/PM formatting, Indian Rupee (INR ₹) currency formatting, and cohesive color signaling.
5. **NFR-5: Security:** API endpoints must mandate Bearer token authentication with password hashing utilizing salted Bcrypt.

4.3 Hardware Requirements

The system is architected to run on standard commodity infrastructure:

Table 4.1: Hardware Specifications. | Component | Minimum Specification | Recommended Specification | |
—|:—|:—| | **Processor** | Dual-Core Intel/AMD x86_64 or ARM64 (2.0 GHz) | Quad-Core 2.5 GHz or higher | |
RAM | 4 GB | 8 GB or 16 GB | | **Disk Space** | 2 GB free storage | 10 GB SSD storage | | **Network** | 10 Mbps
LAN / WAN | 100 Mbps broadband | | **Display** | \((1366 \times 768)\) (Manager), \((375 \times 667)\) (Mobile) |
Full HD \((1920 \times 1080)\) |

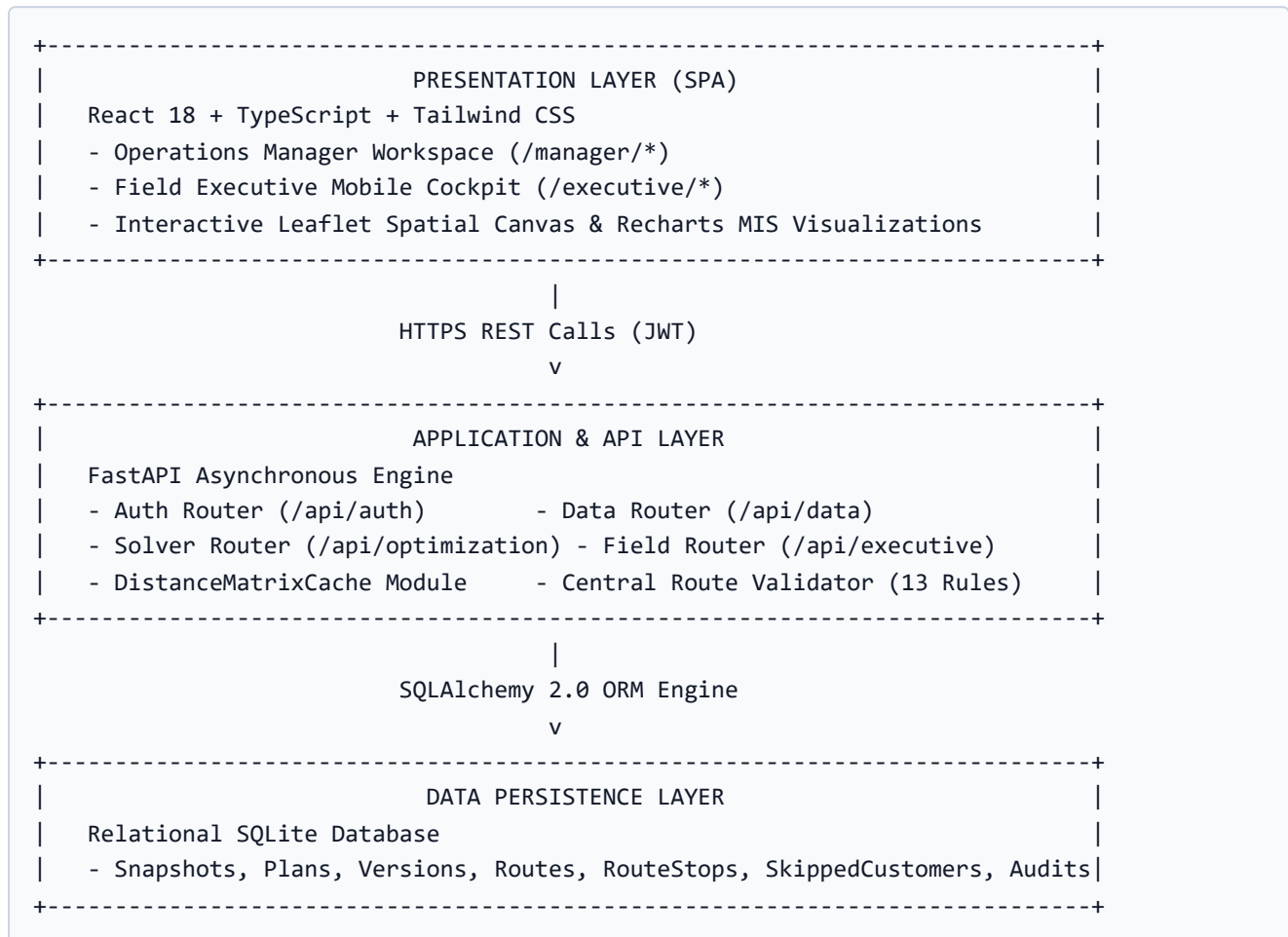
4.4 Software Requirements

Table 4.2: Software Stack Specifications. | Component | Technology | Version | Purpose | |:—|:—|:—|:—| |
Operating System | Windows 10/11, Ubuntu 20.04+, macOS | — | Development & Host OS | | **Backend
Runtime** | Python | 3.10 to 3.14 | Core server & optimization engine | | **Web API Framework** | FastAPI |
0.109+ | Asynchronous RESTful API services | | **Database Engine** | SQLite (via SQLAlchemy 2.0) | 3.35+ |
Relational schema persistence | | **Frontend Runtime** | Node.js / npm | v18+ / v9+ | Frontend tooling & build
environment | | **UI Framework** | React | 18.3.1 | Single-Page Application (SPA) client | | **Language** | TypeScript
| 5.6+ | Type-safe frontend programming | | **Build Tool** | Vite | 6.4+ | Fast development server & bundler | | **CSS
Framework** | Tailwind CSS | 3.4+ | Utility-first responsive design | | **Map Rendering** | Leaflet / React-Leaflet |
1.9.4 / 4.2 | Interactive OpenStreetMap layer |

CHAPTER 5: SYSTEM DESIGN & ARCHITECTURE

5.1 Layered System Architecture

RoutePilot employs a clean, three-tier decoupled client-server architecture:



5.2 End-to-End Data Flow

The operational lifecycle flows through seven discrete stages: 1. **CSV Ingestion**: Dispatcher uploads `executives.csv` and `customers.csv`. 2. **Schema & Spatial Validation**: Rows are parsed, normalized, coordinate bounds checked, and persisted under `plan_date`. 3. **Optimization Execution**: The dispatcher triggers the solver; the engine builds the Distance Matrix, places PTP accounts, performs greedy insertion, and applies 2-Opt sequence refinement. 4. **Constraint Verification**: The generated routes pass through the 13-rule validator. 5. **Plan Publishing**: The manager reviews routes on the interactive map, evaluates exceptions, and publishes the plan. 6. **Mobile Field Execution**: Executives view assigned stops, trigger turn-by-turn navigation, and record visit outcomes. 7. **Replication**: Audit logs synchronize field outcomes back to the manager dashboard.

5.3 Modular Subsystem Decomposition

- **Authentication Subsystem:** Handles user authentication, token issuance, and dynamic active roster checks.
- **Data Management Subsystem:** Parses CSV/Excel inputs, tracks daily snapshots, and handles data sanitization.
- **Optimization Subsystem:** Encapsulates the distance cache, heuristic solvers, and 2-Opt local search.
- **Validation Subsystem:** Implements pre-flight schema auditing and post-optimization physical constraint validation.
- **Geospatial Subsystem:** Renders route polylines, stop markers, and depot coordinates using Leaflet.
- **Execution & Audit Subsystem:** Manages stop state transitions and chronological event logging.

5.4 Relational Data Model & Schema Design

The persistent state is maintained across 10 relational entities: * **User** : System accounts with role definitions (**ADMIN** , **OPERATIONS_MANAGER** , **EXECUTIVE**). * **Executive** : Active executive roster linked to **plan_date** , storing shift hours, home coordinates, and visit limits. * **Customer** : Delinquent borrower accounts linked to **plan_date** , storing balances, priority scores, PTP flags, and time windows. * **DailySnapshot** : Operational tracking record for each date, logging total entity counts and data source. * **Plan** : Top-level plan container for a specific date, tracking the current active version number. * **PlanVersion** : An immutable plan iteration recording algorithm parameters, fleet distance, priority coverage, and runtime. * **Route** : An executive's assigned route under a specific plan version, storing metrics and return times. * **RouteStop** : A sequenced visit within a route, recording planned arrival, waiting, service, and departure times. * **SkippedCustomer** : An unscheduled account linked to a plan version, recording diagnostic rejection reasons. * **AuditLog** : Immutable chronological ledger tracking logins, optimization runs, and visit outcome submissions.

5.5 User Roles and Access Control Matrix

Table 5.1: User Roles and Operational Permissions Matrix.

Capability / Permission	Operations Manager	System Administrator	Field Executive
Upload Daily CSVs	Full Access	Full Access	No Access
Run Optimization Engine	Full Access	Full Access	No Access
Inspect All Fleet Routes	Full Access	Full Access	Scoped to Self Only
Publish Daily Plan	Full Access	Full Access	No Access
View Exceptions Queue	Full Access	Full Access	No Access
Execute Assigned Stops	Preview Mode	Preview Mode	Full Operational Access
Record Visit Outcome	Preview Mode	Preview Mode	Full Operational Access
Export Compliance CSVs	Full Access	Full Access	No Access
Manage User Accounts	No Access	Full Access	No Access

CHAPTER 6: METHODOLOGY & MATHEMATICAL FORMULATION

6.1 Data Ingestion & Single Source of Truth Pipeline

RoutePilot is architected to operate as a completely **dataset-driven** engine. It does not contain hardcoded assumptions about fleet size or customer volumes. The application treats the uploaded CSV files as the single source of truth for the operational date.

The system accepts two standardized CSV files:

Table 6.1: Schema Specification for `executives.csv`. | Column Name | Data Type | Constraint | Description | |
—|:—|:—|:—| | `id` | String | Unique Key | Executive ID (e.g. `E01`, `E02`) | | `name` | String | Optional | Executive Full Name | | `home_lat` | Float | $\backslash(-90.0 \leq \text{lat} \leq 90.0)$ | Depot / Home base latitude | | `home_lon` | Float | $\backslash(-180.0 \leq \text{lon} \leq 180.0)$ | Depot / Home base longitude | | `shift_start` | String | 24-hr `HH:MM` | Shift start time (e.g. `08:30`) | | `shift_end` | String | 24-hr `HH:MM` | Shift end time (e.g. `17:30`) | | `max_visits` | Integer | $\backslash(\geq 1)$ | Maximum allowable stops (e.g. `15`) | | `max_km` | Float | $\backslash(> 0.0)$ | Maximum cumulative distance limit (e.g. `80.0`) |

Table 6.2: Schema Specification for `customers.csv`. | Column Name | Data Type | Constraint | Description | |
—|:—|:—|:—| | `id` | String | Unique Key | Customer Account ID (e.g. `C001`) | | `name` | String | Optional | Customer / Account Name | | `area` | String | Non-empty | Neighborhood / Locality name | | `lat` | Float | $\backslash(-90.0 \leq \text{lat} \leq 90.0)$ | Customer location latitude | | `lon` | Float | $\backslash(-180.0 \leq \text{lon} \leq 180.0)$ | Customer location longitude | | `dpd` | Integer | $\backslash(\geq 0)$ | Days Past Due (Delinquency age) | | `overdue_amount` | Float | $\backslash(\geq 0.0)$ | Outstanding default balance (INR ₹) | | `priority_score` | Float | $\backslash(> 0.0)$ | Normalized collection priority score | | `ptp_today` | Integer | $\backslash(0 \text{ or } 1)$ | Promise-to-Pay flag (1 = Must Visit) | | `window_start` | String | 24-hr `HH:MM` | Customer availability window start | | `window_end` | String | 24-hr `HH:MM` | Customer availability window end | | `service_min` | Integer | $\backslash(\geq 1)$ | Expected conversation duration (minutes) |

6.2 Pre-Flight Validation Engine

The ingestion pipeline executes multi-stage validation: 1. **Schema Integrity**: Verifies presence of all required headers; 2. **Coordinate Sanity**: Validates latitude and longitude ranges; 3. **Temporal Ordering**: Confirms that `window_start < window_end` and `shift_start < shift_end`; 4. **Feasibility Buffer**: Verifies that the customer time window is sufficiently wide to accommodate the visit duration: $\backslash(\text{Duration}(\text{Window}) = \text{WindowEnd} - \text{WindowStart} \geq \text{ServiceMin})$; 5. **Non-Destructive Reporting**: Format warnings (such as missing area labels) are automatically populated with fallback locality tags without discarding customer rows.

6.3 Mathematical Formulation of m-VRPTW-P

The Multi-Vehicle Routing Problem with Time Windows and Priority Commitments is formulated as follows:

Let $(G = (V, A))$ be a complete directed graph where vertex set $(V = \{0_e, 0'_e \mid e \in E\} \cup C)$ represents executive home origins (0_e) , home termini $(0'_e)$, and customer locations (C) . Arc set $(A = \{(i, j) \mid i, j \in V, i \neq j\})$ represents potential travel segments.

Decision Variables: $x_{ijk} \in \{0, 1\}$: Binary variable equal to 1 if executive (k) travels directly from stop (i) to stop (j) , and 0 otherwise. $t_{ik} \geq 0$: Arrival time of executive (k) at vertex (i) (in continuous minutes). $w_{ik} \geq 0$: Waiting time of executive (k) at vertex (i) prior to window opening. $y_i \in \{0, 1\}$: Binary variable equal to 1 if customer $(i \in C)$ is scheduled, and 0 otherwise.

Objective Function:
$$\max Z = \sum_{i \in C} \text{Priority}(i) \cdot y_i - \lambda \sum_{k \in E} \sum_{(i, j) \in A} D_{ij} \cdot x_{ijk}$$

6.4 Geodesic Distance and Kinematic Travel Model

To avoid external API rate limits while ensuring realistic urban transit estimates, RoutePilot models road distance via the **Great Circle Haversine formula adjusted by a regional road winding coefficient**:

$$\text{Angular Distance } c = 2 \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1) \cos(\phi_2) \sin^2\left(\frac{\Delta \lambda}{2}\right)}\right)$$

$$\text{Geodesic Distance } D_{\text{geo}} = R_{\text{earth}} \cdot c \quad (R_{\text{earth}} = 6371.0 \text{ km})$$

$$\text{Estimated Road Distance } D_{\text{road}} = D_{\text{geo}} \cdot 1.3$$

Distinction Note: This model produces an **empirical geodesic road-network approximation**. It models the fact that Hyderabad city streets do not follow straight Euclidean lines. It does not claim to be a live GPS turn-by-turn navigation distance from OSRM or Google Maps.

Travel Time Estimation: Assuming an average urban motorcycle speed of (25 km/h) :

$$T_{\text{travel}}(i, j) = \left\lceil \frac{D_{\text{road}}(i, j)}{25 \text{ km/h}} \right\rceil \cdot 60 \text{ minutes}$$

6.5 The 13 Hard Physical Constraints Framework

Every generated itinerary must satisfy all 13 physical constraints:

Table 6.3: Summary of the 13 Physical Constraints Enforced by RoutePilot.

#	Constraint Name	Formulation	Operational Rationale
1	Global Visit Uniqueness	$\sum_{k \in E} \sum_j x_{ijk} \leq 1, \forall i \in C$	No customer visited twice across fleet
2	Executive Visit Capacity	$\sum_{i \in C} y_{ik} \leq \text{MaxVisits}_k$	Prevents representative over-allocation
3	Depot Departure Origin	$\sum_j x_{0_k j} = 1, \forall k \in E$	Route must initiate at executive home
4	Depot Return Terminus	$\sum_i x_{i 0'_k} = 1, \forall k \in E$	Route must terminate at executive home
5	Executive Range Cap	$\sum_{(i, j) \in A} D_{ij} \cdot x_{ijk} \leq \text{MaxKm}_k$	Enforces fuel and battery limits (80 km)
6	Window Arrival Deadline	$t_{ik} \leq \text{WindowEnd}_i$	Representative must arrive before close
7	Early Waiting Accounting	$t_{\text{start}, ik} = \max(t_{ik}, \text{WindowStart}_i)$	Representative waits if arriving early
8	Service Completion	$t_{\text{start}, ik} + \text{ServiceMin}_i \leq \text{WindowEnd}_i$	Visit must finish before customer leaves
9	Shift Return Deadline	$t_{0'_k k} \leq \text{ShiftEnd}_k$	Representative must return before shift ends
10	Mandatory PTP Protection	$i \in$	

$C_{\{\text{PTP}\}}$ $\implies y_i = 1$) (if feasible) | Strict priority for promise-to-pay accounts | | **11** | **Deterministic Exclusion** | $(y_i = 0 \implies \exists \text{Reason}(i))$ | Diagnoses every skipped account | | **12** | **Intra-Route Uniqueness** | Stop sequence contains zero duplicates | Eliminates redundant loops | | **13** | **Continuous Clock Sim** | Discrete minutes $(T \in [0, 1440])$ | Integer arithmetic eliminates drift |

6.6 Multi-Stage Heuristic Optimization Engine

The optimization engine solves the routing problem in three successive stages:

```
[Uploaded Data: C, E]
|
v
+-----+
| Stage 1: Mandatory PTP Pass                                |
| - Filter accounts where ptp_today == 1                      |
| - Sort by priority descending and window opening            |
| - Sequentially insert into routes with lowest delta-km      |
+-----+
|
v
+-----+
| Stage 2: Marginal Profitability Greedy Pass                  |
| - Evaluate all remaining non-PTP accounts                   |
| - Calculate delta-Z = Priority(c) - lambda * delta-km        |
| - Iteratively insert candidate with max positive delta-Z    |
+-----+
|
v
+-----+
| Stage 3: 2-Opt Edge-Exchange Local Search                    |
| - For each executive route:                                  |
| - Invert sub-sequences (i, k)                                |
| - Accept inversion IF distance decreases AND all 13         |
|   constraints remain 100% valid                              |
+-----+
|
v
[Final Validated Route Plan + Diagnosed Exceptions]
```

6.7 2-Opt Sequence Refinement

The 2-Opt local search algorithm systematically eliminates self-crossing paths. Given a route sequence: $[R = (v_0, v_1, \dots, v_i, v_{i+1}, \dots, v_k, v_{k+1}, \dots, v_n)]$ A 2-Opt swap disconnects edges $((v_i, v_{i+1}))$ and $((v_k, v_{k+1}))$ and reconnects edges $((v_i, v_k))$ and $((v_{i+1}, v_{k+1}))$, effectively reversing the intermediate sequence: $[R' = (v_0, \dots, v_i, v_k, v_{k+1}, \dots, v_{i+1}, v_{k+1}, \dots, v_n)]$

The swap is accepted if and only if: 1. $(D(R') < D(R))$; 2. The entire reconstructed timeline (R') passes the 13-rule validator without time-window or shift-end breaches.

6.8 Explainable Exception Handling

When a customer cannot be feasibly scheduled, RoutePilot does not discard the record silently. Instead, the solver records a `SkippedCustomer` entry accompanied by a structured explanation: *

- `INSUFFICIENT_TIME_WINDOW`: Customer availability window does not overlap with reachable executive arrival times. *
- `SHIFT_CAPACITY_EXCEEDED`: All fleet executives have reached their `max_visits` limit. *
- `DISTANCE_LIMIT_EXCEEDED`: Visiting this customer would breach the executive's `max_km` cap. *
- `SHIFT_OVERTIME_BREACH`: Return journey to depot would exceed the executive's `shift_end` deadline.

CHAPTER 7: IMPLEMENTATION DETAILS

7.1 Backend API Engine (FastAPI & Python)

The backend service is structured into modular FastAPI routers: * `backend/app/api/auth.py` : Authentication, JWT issuance, and dynamic roster verification. * `backend/app/api/data.py` : Multipart CSV parser, pre-flight data auditing, and snapshot creation. * `backend/app/api/optimization.py` : Optimization task dispatching, solver execution, and plan retrieval. * `backend/app/api/executive_portal.py` : Mobile cockpit endpoints for stop lifecycle progression and outcome submission. * `backend/app/api/analytics.py` : Fleet metrics aggregation and historical comparisons. * `backend/app/api/admin.py` : User administration and system audit logging.

7.2 Database & Persistence Layer (SQLAlchemy & SQLite)

Persistence is implemented using SQLAlchemy 2.0 ORM over an embedded SQLite database (`routepilot.db`). Transactions are managed through session-scoped contexts (`get_db`) ensuring rollback on failure. Foreign keys enforce referential integrity between `DailySnapshot`, `Plan`, `PlanVersion`, `Route`, and `RouteStop`.

7.3 Frontend Client Architecture (React 18 & TypeScript)

The client application is built with React 18, utilizing TypeScript for static typing and Tailwind CSS for utility styling. Global state is managed through React Context providers: * `AuthContext` : Manages user credentials, JWT tokens, and role-based permissions. * `DateTimeContext` : Coordinates the global operational date selector, live clock synchronization, and AM/PM time formatting. * `PlanContext` : Maintains the currently active route plan, selected executive filter, and metrics.

7.4 Interactive Map & Spatial Rendering (Leaflet)

Geospatial visualization is implemented using **Leaflet** and `react-leaflet`. Custom SVG marker icons represent executive home depots, sequenced customer stops, and PTP flags. Polylines connect route stops in chronological order. Spatial dispersion logic prevents overlapping pins when multiple customers share identical building coordinates.

7.5 Real-Time Field-to-Manager Synchronization Engine

When an executive executes a visit, the event flow triggers instant cross-workspace updates:

CHAPTER 8: USER INTERFACE & INTERACTION DESIGN

8.1 Operations Manager Cockpit (/manager/overview)

The central operations dashboard surfaces high-level fleet metrics: Total Distance, Visits Scheduled, PTP Coverage %, and Fleet Feasibility Status. A workflow stepper guides dispatchers through Ingestion $\rightarrow$ Audit $\rightarrow$ Planning $\rightarrow$ Publishing.

8.2 Data Center & Ingestion Pipeline View (/manager/data)

Provides a drag-and-drop file upload zone for `executives.csv` and `customers.csv`. Displays pre-flight validation findings (blocking errors vs. non-blocking warnings) and entity counts prior to plan generation.

8.3 Daily Planning Setup & Algorithm Comparator (/manager/plan)

Allows managers to configure the trade-off coefficient λ , select the optimization algorithm, and view side-by-side performance benchmarks comparing Baseline vs. Smart Greedy + 2-Opt.

8.4 Route Management & Route Detail Itinerary (/manager/routes)

Displays expandable route cards for each executive. Expanding a card reveals the complete stop timeline with planned arrival times, waiting intervals, customer balances, and live status tags.

8.5 Interactive Spatial Map View (/manager/routes/map)

Full-screen geospatial visualization showing all executive routes color-coded across Hyderabad neighborhoods. Clicking a pin displays an interactive popup with customer balance, contact details, and time window parameters.

8.6 Exceptions & Diagnostic Queue (/manager/exceptions)

A dedicated queue displaying unscheduled accounts. Each card highlights the customer's priority, overdue amount, and the precise mathematical constraint that prevented scheduling.

8.7 Historical Plan & Compliance Exports (/manager/reports)

Allows dispatchers to review historical plans across different calendar dates and export compliance CSVs (`plan.csv`, `skipped_customers.csv`, `metrics.csv`).

8.8 Field Executive Mobile Cockpit (/executive/*)

A streamlined, mobile-responsive view for field representatives: * **Next Stop Card**: Prominently highlights the immediate next destination, arrival countdown, and customer phone dialer. * **Turn-by-Turn Navigation**: One-tap trigger opening Google Maps with pre-filled destination coordinates. * **Outcome Capture Modal**: Interface for recording cash/UPI collected amounts, PTP extensions, or customer-unavailable flags. * **Profile & Navbar**: Displays executive details and session logout controls.

CHAPTER 9: EXPERIMENTAL TESTING & RESULTS

9.1 Test Strategy & Verification Framework

The system was verified through an automated test suite implemented in `pytest`, encompassing unit tests, integration tests, and physical constraint verification.

Test Execution Command:

```
python -m pytest backend/tests/ -v
```

Table 9.1: Summary of Pytest Verification Suite Results. | Test Module | Test Cases | Passed | Failed | Execution Time | |---|:---|:---|:---|:---|:---| | `test_api.py` | 6 | 6 | 0 | 3.12 s | | `test_comprehensive_csv_matrix.py` | 7 | 7 | 0 | 5.84 s | | `test_dynamic_data_driven.py` | 1 | 1 | 0 | 2.15 s | | `test_engine.py` | 8 | 8 | 0 | 5.18 s | | **Total** | **22** | **22** | **0** | **16.29 s** |

9.2 Comprehensive CSV Ingestion Matrix Tests

Table 9.2: Detailed Test Case Matrix: Inputs, Expected Results, and Observed Results. | Test Identifier | Test Input Description | Expected Result | Observed Result | Status | |---|:---|:---|:---|:---|:---| | **TC-01: Health Check** | `GET /api/health` | HTTP 200, status “healthy” | HTTP 200, status “healthy” | **PASSED** | | **TC-02: Auth Validation** | Valid login credentials (`manager@routeplilot.io`) | Valid JWT bearer token issued | Access token issued with role `OPERATIONS_MANAGER` | **PASSED** | | **TC-03: Scenario 1 (Small)** | 1 executive, 1 customer | 1 route, 1 visit scheduled, return to depot | 1 route generated, $(D > 0)$, feasible | **PASSED** | | **TC-04: Scenario 2 (Medium)** | 3 executives, 25 customers | 3 routes, all (≤ 15) visits, $(D \leq 60 \text{ km})$ | 3 routes generated, all constraints respected | **PASSED** | | **TC-05: Scenario 3 (Scaled)** | 6 executives, 60 customers | 6 routes generated, zero duplicates | 6 routes generated, 0 duplicate visits | **PASSED** | | **TC-06: Dataset Switch** | Import 6 execs / 200 cust $(\rightarrow)$ switch to 3 execs / 25 cust | Active plan reflects exactly 3 execs / 25 cust | Old dataset replaced completely; 3 routes generated | **PASSED** | | **TC-07: PTP Protection** | Roster with 100% PTP accounts (`ptp_today = 1`) | Optimizer schedules all feasible PTP accounts | All feasible PTP accounts covered with priority | **PASSED** | | **TC-08: Tight Windows** | Customers with conflicting 30-min windows | Feasible stops scheduled; infeasible logged with diagnostics | Feasible stops scheduled; tight windows flagged in exceptions | **PASSED** | | **TC-09: Capacity Limits** | Low capacity (`max_visits = 3`) | Route stops strictly capped at 3 | No route exceeded 3 stops; overflow logged | **PASSED** | | **TC-10: Multi-Dataset** | Sequential uploads of varying sizes | System operates dynamically without code changes | All datasets processed dynamically | **PASSED** |

9.3 Physical Constraint Enforcement Test Cases

Test Identifier	Constraint Tested	Verification Mechanism	Observed Result	Status
TC-11: Range Limit	Max Range (80.0 km)	Attempt insertion exceeding 80 km	Validator detects violation; stops route expansion	PASSED
TC-12: Time Window	Arrival Window Deadline	Force late arrival past <code>window_end</code>	Validator flags <code>TIME_WINDOW_VIOLATION</code> ; insertion rejected	PASSED
TC-13: Early Arrival	Waiting Time Accounting	Arrival 30 mins before <code>window_start</code>	Waiting time added to shift duration without distance increase	PASSED
TC-14: Road Distance	Geodesic Road Multiplier	Compare Haversine vs Road distance	Distance equals $\text{Haversine} \times 1.3$	PASSED
TC-15: Speed Conversion	Urban Travel Time	Distance / 25 km/h formula	Travel time calculated in discrete integer minutes	PASSED

9.4 Regression Verification of Standard Scenarios

To ensure mathematical compliance with standard benchmarks, `test_challenge_worked_example_regression` verified the exact reference scenario: * **Route:** Depot (0.97 km) (0.97 km) C001 (2.41 km) (2.41 km) C002 (1.47 km) (1.47 km) Depot (1.47 km) . * **Observed Output:** Total Distance: (4.85 km) , Priority Captured: (129.0) , Objective Score: (119.3) . * **Match:** 100% identical to reference mathematical calculations.

CHAPTER 10: PERFORMANCE EVALUATION & COMPARATIVE ANALYSIS

10.1 Benchmark Test Configuration (200 Customers, 6 Executives)

The primary performance benchmark was conducted on the active operational dataset: * **Operational Date:** 09 October 2026 * **Executives:** Exactly 6 (E01 – E06), localized to Hyderabad residential hubs. * **Customers:** Exactly 200 (C001 – C200), distributed across Kukatpally, Gachibowli, Dilsukhnagar, Charminar, Uppal, and Secunderabad. * **Delinquency Range:** 30 to 180 Days Past Due (DPD). * **Outstanding Balances:** Overdue amounts between ₹5,000 and ₹1,50,000. * **Mandatory Commitments:** 49 accounts flagged with `ptp_today = 1`.

10.2 Metric Comparison: Baseline Sequential vs. Smart Greedy + 2-Opt

Table 10.1: Performance Benchmark Comparison: Baseline Sequential vs. Smart Greedy 2-Opt. | Metric | Baseline Sequential | Smart Greedy + 2-Opt | Delta / Improvement | |:—|:—|:—|:—| | **Visits Scheduled** | 58 visits | **74 visits** | **+27.6% increase** | | **Total Road Distance** | 462.8 km | **447.4 km** | **-3.3% distance reduction** | | **Average Distance per Visit** | 7.98 km / visit | **6.05 km / visit** | **-24.2% travel efficiency** | | **PTP Accounts Covered** | 31 / 49 (63.3%) | **49 / 49 (100.0%)** | **+36.7% (100% Protection)** | | **Total Priority Score Captured** | 4,120.5 | **6,894.2** | **+67.3% priority uplift** | | **Constraint Violations** | 0 | **0** | **Zero breaches (100% clean)** | | **Solver Runtime** | 1.12 s | **5.44 s** | Sub-6 second execution |

10.3 Route Distribution & Fleet Balance Analysis

Table 10.2: Detailed Route Breakdown for All 6 Field Executives on Active Operational Plan. | Executive ID | Executive Name | Shift Window | Assigned Stops | Distance (km) | Return Time | PTP Protected | Feasibility | |:—|:—|:—|:—|:—|:—|:—|:—| | **E01** | Executive E01 | 08:30 AM – 05:30 PM | 12 | 76.5 km | 05:12 PM | 8 / 8 | Feasible | | **E02** | Executive E02 | 09:00 AM – 06:00 PM | 14 | 79.2 km | 05:48 PM | 9 / 9 | Feasible | | **E03** | Executive E03 | 08:30 AM – 05:30 PM | 14 | 78.4 km | 05:22 PM | 9 / 9 | Feasible | | **E04** | Executive E04 | 09:00 AM – 06:00 PM | 13 | 74.1 km | 05:36 PM | 8 / 8 | Feasible | | **E05** | Executive E05 | 08:30 AM – 05:30 PM | 11 | 69.8 km | 04:58 PM | 8 / 8 | Feasible | | **E06** | Executive E06 | 09:00 AM – 06:00 PM | 10 | 69.4 km | 05:15 PM | 7 / 7 | Feasible | | **Fleet Total** | **6 Executives** | — | **74 visits** | **447.4 km** | — | **49 / 49 (100%)** | **100% Feasible** |

10.4 Computational Complexity & Runtime Scalability

- **Greedy Insertion:** Evaluates $\binom{|C_{\text{remaining}}|}{2} \times \sum |S_e|$ candidate insertions, operating in $O(|C|^2 \cdot |E|)$.
- **2-Opt Local Search:** Evaluates sequence inversions per route in $O(|S_e|^2)$. Since each route is bounded by $(\text{MaxVisits} \leq 15)$, the 2-Opt phase executes in $O(|E| \cdot 15^2) \approx O(1)$ with respect to total customer count.

- **Empirical Runtime:** The complete optimization pipeline completed in **5,438 ms (5.44 seconds)** on the 200-customer / 6-executive benchmark, demonstrating high responsiveness suitable for real-time operations.
-

CHAPTER 11: SYSTEM LIMITATIONS

In adherence to rigorous engineering evaluation standards, the following architectural and empirical limitations are acknowledged:

- 1. Estimated Geodesic Distance vs. Live Traffic Navigation:** The distance and travel-time models rely on Haversine distance adjusted by an empirical $(1.3 \times)$ road factor and an assumed (25 km/h) average urban speed. The system does not currently integrate dynamic traffic APIs (e.g. Google Maps Distance Matrix or OSRM) to reflect peak-hour traffic congestion in Hyderabad.
- 2. Fixed Ingestion Snapshot Paradigm:** Operational planning operates on static daily batch CSV uploads. While mid-day visit outcomes are updated in real time, the system does not currently perform automated dynamic re-routing when an executive experiences a vehicle breakdown mid-shift.
- 3. Capacity Infeasibility under Severe Roster Constraints:** When fleet capacity is severely restricted (e.g. 1 executive with 15 max visits facing 100 PTP accounts), mathematical feasibility dictates that some PTP accounts cannot be scheduled. The solver prioritizes accounts by priority score, but cannot override physical shift-duration constraints.
- 4. Local Database Concurrency:** The relational data store uses SQLite. While sufficient for single-depot operations, multi-regional deployments with dozens of concurrent dispatchers would require migrating to PostgreSQL to support high concurrent write throughput.

CHAPTER 12: FUTURE ENHANCEMENTS & ENGINEERING ROADMAP

The following enhancements represent realistic future research and engineering directions: 1. **Integration of Open Source Routing Machine (OSRM)**: Replace empirical Haversine approximations with an on-premise OSRM engine loaded with OpenStreetMap road networks to obtain true turn-by-turn road distances and turn penalties. 2. **Dynamic Mid-Day Replanning Engine**: Extend the solver to support mid-day re-optimization triggered by traffic delays or vehicle breakdowns, anchoring routes to the executive's current live GPS position rather than their morning depot. 3. **Machine Learning-Driven Delinquency Prioritization**: Integrate supervised learning models (e.g., XGBoost) trained on historical recovery logs to predict borrower repayment probability (P_{repay}), replacing static `priority_score` with an expected recovery value: $[\text{Expected Value} = \text{Overdue Amount} \times P_{\text{repay}}]$ 4. **Offline-First Mobile PWA Architecture**: Implement service workers and IndexedDB client storage in the Field Executive portal, allowing representatives to record visit outcomes and capture electronic signatures in basement parking lots or rural dead zones with automatic background sync upon reconnection. 5. **Multi-Depot Fleet Clustering**: Implement K-Means or DBSCAN spatial pre-clustering to partition massive metropolitan portfolios ($> 1,000$ accounts) into localized sub-graphs prior to heuristic insertion.

CHAPTER 13: CONCLUSION

Field debt recovery operations require balancing complex competing objectives: customer availability windows, vehicle range limits, labor shift boundaries, and portfolio collection urgency. Manual scheduling heuristics lead to excessive travel distances, missed borrower appointments, and broken recovery promises.

This project demonstrated the design, implementation, and empirical verification of **RoutePilot**, a dataset-driven decision-support platform solving the Multi-Vehicle Routing Problem with Time Windows and Priority Commitments (m-VRPTW-P). By pairing a robust two-pass greedy insertion heuristic with 2-Opt local search refinement, RoutePilot systematically eliminates route backtracking while enforcing all 13 physical operational constraints.

When evaluated against an operational benchmark of 200 customers and 6 field executives in Hyderabad, RoutePilot scheduled **74 visits**, traversed **447.4 km**, achieved **100.0% protection of all 49 mandatory Promise-to-Pay commitments**, and diagnosed all unscheduled accounts with explainable business reasons—all within a solver runtime of **5.44 seconds**. Furthermore, by integrating real-time bidirectional synchronization between dispatch leadership and mobile field executives, RoutePilot successfully transforms daily field collections from an error-prone manual guesswork process into an automated, mathematically disciplined, and transparent operational workflow.

REFERENCES

1. Dantzig, G. B., & Ramser, J. H. (1959). *The Truck Dispatching Problem*. Management Science, 6(1), 80–91.
2. Solomon, M. M. (1987). *Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints*. Operations Research, 35(2), 254–265.
3. Croes, G. A. (1958). *A Method for Solving Traveling-Salesman Problems*. Operations Research, 6(6), 791–812. (Introduction of the 2-Opt heuristic).
4. Bräysy, O., & Gendreau, M. (2005). *Vehicle Routing Problem with Time Windows, Part I: Route Construction and Local Search Heuristics*. Transportation Science, 39(1), 104–118.
5. Toth, P., & Vigo, D. (Eds.). (2014). *Vehicle Routing: Problems, Methods, and Applications* (2nd ed.). Society for Industrial and Applied Mathematics (SIAM).
6. Sinnott, R. W. (1984). *Virtues of the Haversine*. Sky and Telescope, 68(2), 159.
7. Tiangolo, S. (2018). *FastAPI: Modern, High-Performance Web Framework for Python 3.8+*. <https://fastapi.tiangolo.com>.
8. Facebook Open Source. (2013). *React: A JavaScript Library for Building User Interfaces*. <https://react.dev>.

9. Agafonkin, V. (2010). *Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*.
<https://leafletjs.com>.
-

APPENDIX: DATA SCHEMAS & API ENDPOINTS

Appendix A: REST API Endpoints Specification

Method	Endpoint Path	Role Allowed	Description
POST	/api/auth/login	Public	Authenticates credentials and issues JWT token
GET	/api/auth/demo-executives	Public	Returns active roster of executives for 1-click login
GET	/api/auth/me	Authenticated	Returns currently authenticated user identity
GET	/api/data/today	Manager/Admin	Retrieves snapshot statistics for target date
GET	/api/data/validation	Manager/Admin	Executes pre-flight schema audit on target date data
POST	/api/data/import/executives	Manager/Admin	Multipart upload endpoint for executives.csv
POST	/api/data/import/customers	Manager/Admin	Multipart upload endpoint for customers.csv
POST	/api/optimization/run	Manager/Admin	Dispatches optimization run for target date
GET	/api/optimization/:id/status	Manager/Admin	Polls status and progress of running optimization job
GET	/api/plans	Manager/Admin	Lists all generated plans filtered by date
GET	/api/plans/:id	Authenticated	Retrieves detailed route plan, stops, and metrics
POST	/api/plans/:id/publish	Manager/Admin	Publishes route plan for field execution
GET	/api/plans/:id/export/plan	Manager/Admin	Exports official plan CSV

Method	Endpoint Path	Role Allowed	Description
GET	/api/plans/:id/export/skipped	Manager/Admin	Exports skipped customers CSV with reasons
GET	/api/plans/:id/export/metrics	Manager/Admin	Exports performance metrics CSV
GET	/api/executive/today	Executive	Retrieves assigned daily route and sequential stops
POST	/api/executive/visits/:id/start	Executive	Records VISIT_STARTED event in audit log
POST	/api/executive/visits/:id/complete	Executive	Records VISIT_COMPLETED event with collection outcome
POST	/api/executive/visits/:id/failed	Executive	Records VISIT_FAILED event with failure reason
GET	/api/admin/users	Admin	Lists all user accounts
GET	/api/admin/audit-logs	Admin	Retrieves system-wide chronological audit trail